

54. Un argument par défaut

Ce qui n'est pas écrit prend la valeur annoncée.

1. Ce qu'il faut comprendre

L'argument par défaut est rempli **à l'appel**, et non dans la fonction — comme en C++. Cela lui permet de dépendre de l'endroit d'où l'on appelle.

Une règle : un argument sans valeur par défaut **ne peut pas suivre** un argument qui en a une. Sinon, à `barre(4, 20)`, on ne pourrait plus deviner lequel manque. Le compilateur le refuse plutôt que de choisir à notre place.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1378 octets** de cartouche.

1. UNE FONCTION — BARRE()

```
void barre(uint8_t ligne, uint8_t combien, uint8_t tuile = 1) {  
    for (uint8_t x = 0; x < combien; x++) poser(x, ligne, tuile);  
}
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

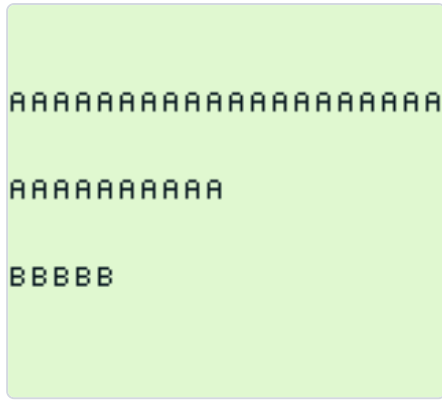
```
int main() {  
    barre(4, 20);  
    barre(8, 10);  
    barre(12, 5, 2);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Trois barres de longueurs différentes ; la dernière d'une autre tuile.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la barre longue fait vingt cases
- ✓ la moyenne s'arrête à dix
- ✓ la courte prend la tuile donnée
- ✓ et s'arrête à cinq

Ensuite : **55. Pourquoi une fonction ne s'appelle pas elle-même** — niveau 7 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++